

Raku++

A from-scratch Raku, hand-written in C++17.

Interpreter + native compiler

runs in the **browser** via WebAssembly

installs modules, **zef-compatible**

Zero third-party dependencies

v3.28.0

— NOT A RAKUDO FORK — SHARES NO CODE WITH IT

One implementation of the **language**, from lexer to native binary.

A hand-written lexer, parser, and tree-walking evaluator for real **Raku** — classes, roles, grammars, regexes, multi-dispatch, junctions, lazy sequences, a bignum tower, Unicode-correct strings, concurrency. It can also **compile** a program to a standalone native executable, and — as **Raku.js** — run client-side in a browser.

- **C++17, no deps**

Hand-rolled bignum, Unicode tables, and regex engine. Builds anywhere CMake and a C++17 compiler run.

- **Interpret or compile**

The same binary tree-walks a script or transpiles it to C++ and compiles native.

- **Measured, not claimed**

Graded test-by-test against Roast, the official Raku specification suite.

— GRADED AGAINST ROAST · MEASURED ON V3.28.0

How much Raku actually runs.

91%

200,504 / ~219,555 individual tests pass, counted against each file's declared plan N — so a file that aborts is charged for every test it never ran.

670 / 1,464

files pass on the strict **all-or-nothing** bar — every assertion in the file has to pass (46%). Up from 594 at v2.0.0, and every one of those 76 files took a real feature, not a tweak.

100%

of S15 — Unicode, strings, NFG — 91,805 / 91,807 assertions. The largest section of the suite, and the finished one: a single file fails two of its own, and nothing else in it does.

Every release is gated on the full suite with **zero fully-passing-file regressions** — and, since v1.5.0, on **performance** too. Seven gates now, and since v3.22.0 **every gate that can fail is proved to detect a planted defect**. Numbers are measured on each tag, never projected; the whole suite runs in ~3½ minutes through a harness written in Raku and run by Raku++ itself.

— THE BREADTH OF WHAT'S IMPLEMENTED

Big-language Raku, most corners lit.

NUMBERS

Exact by default

Arbitrary-precision `Int`,
exact `Rat` — `0.1+0.2-0.3`
is exactly `0` — `Num`,
`Complex`, allomorphs.

OBJECTS

class · role · grammar

Multi-dispatch,
`BUILD` / `TWEAK`, mixins,
`augment`, full `.^`
introspection, `.wrap`.

REGEX

Grammars that parse

Backtracking engine, Unicode
classes, `token` / `rule`,
`.parse` with action classes &
`make`.

SIGNATURES

multi & proto

Type / literal / `where`
constraints, destructuring,
coercion types, `callsame`.

FUNCTIONAL

Lazy & higher-order

Closures, `*` `WhateverCode`,
junctions, `gather` / `take`,
feeds, `1,2,*,*...Inf`.

METAPROG

Your own operators

User ops in all six categories
with working precedence
traits and meta-operators over
them — and **RakuAST**:
`.AST`, `.DEPARSE`, `.EVAL`,
`visit-children`.

CONCURRENCY

Real OS threads

`start` / `await`,
`Supply` / `react` / `whenever`,
`Channel`, atomics; true
parallelism **on by default**
since v3.0.0 (`RAKUPP_GIL=1`
opts back out).

UNICODE

Grapheme-correct

UAX #29 (emoji ZWJ),
NFC/NFD/NFKC/NFKD, UCA
collation — generated
UCD/UCA 17.0 tables.

— ONE SOURCE, FIVE TARGETS

Four ways to run — and a fifth in the browser.

`rakupp prog.raku`

Interpret — the default; covers the whole language.

`--bundle -o prog`

Standalone binary that re-parses the source at startup.

`--aot -o prog`

Standalone binary embedding the already-parsed AST.

`--exe -o prog`

Transpile to C++ and compile native — anything unsupported falls back to `--bundle` automatically.

`Raku.js → WASM`

In the browser — same semantics as native, client-side, no server, with an embeddable playground.

Opt-in, off by default: `--precomp-modules=on` caches the parsed form of everything a program uses – use XML (10 files, 1,110 lines) goes 16.0 ms → 5.7 ms cold-to-warm. Nothing is written to disk until you ask.

— INTERPRETER · × VS RAKUDO V2026.08 · BOTH NATIVE ARM64, INTERLEAVED ON ONE M3

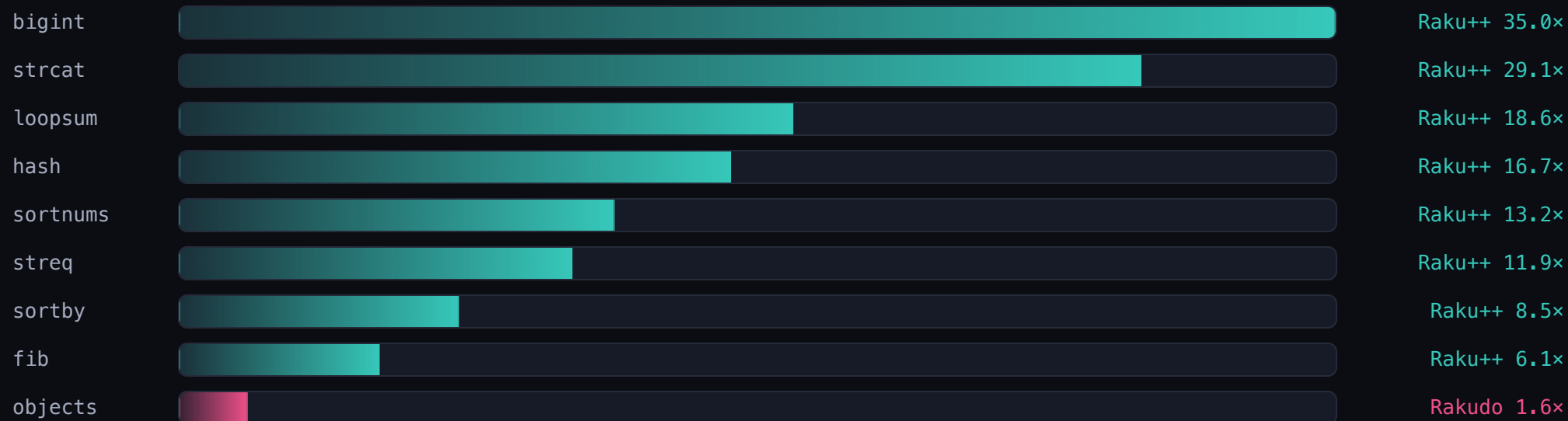
Fast to start, fast to run.



3.0 ms cold start interpreting, 2.5 ms compiled, against Rakudo's 75.6 – a tiny native binary with no VM to spin up. Interpreted, Raku++ leads **eleven of the fifteen** kernels and is *level* on three more – fib, streq and rats, all within 5% of a native arm64 Rakudo. It loses one, and that one is on the next slide too.

— RAKUPP --EXE · TRANSPILED TO C++, COMPILED NATIVE · × VS RAKUDO V2026.08

Compiled, it wins **fourteen** of fifteen.



--exe strips the tree-walk, and fourteen kernels land **1.5×** to **35.0×** ahead of Rakudo. The fifteenth is objects – class, has, method dispatch, the shape most real Raku is written in – where Rakudo's spesh leads by **1.6×**. That kernel is in the set *because* leaving it out would have hidden it; adding it is what found the problem. Anything the compiler can't lower falls back to bundling automatically.

— EVERY DISTRIBUTION IN THE ECOSYSTEM, RUN AGAINST THE ENGINE

The whole ecosystem, **as the test.**

824 / 2,530

distributions **install and pass their own test suites** — the files their authors wrote, not an API probe. 746 one release earlier.

274

never reached their own tests at all: a **dependency failed first**. That cohort is the lever, not the long tail.

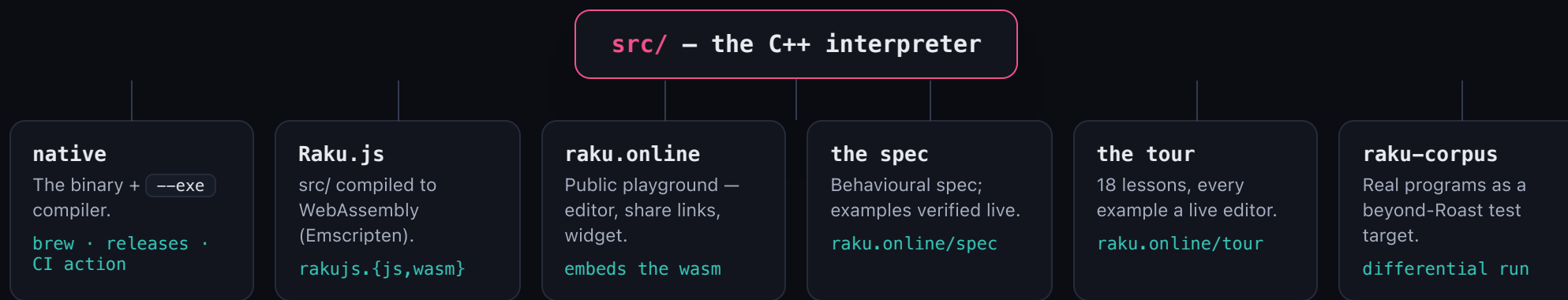
1,000

is the standing target, and the plan is ordered by which failures unblock the most dists downstream.

The sweep is not a scoreboard — it is **the worklist**. What fails decides what gets built next, and nothing is special-cased for a module: the fixes land in the parser, the runtime and dispatch, which is why chasing one distribution keeps paying out in others. Every distribution, with how it ran, is browsable at `raku.online/modules/ecosystem`.

— ONE INTERPRETER BEHIND EVERY SURFACE

A small constellation, one source of truth.



Everything downstream ships the *same* interpreter — the `wasm`, the sub-sites, the Homebrew tap, and the release binaries all trace back to `src/`. Neither the spec nor the tour hosts an engine of its own, so both inherit a new one for free the moment the playground redeploys.

— MID-SIZE PROGRAMS, EACH LEANING ON A DIFFERENT PART OF THE LANGUAGE

Real programs, written **in** Raku.

Scheme

A Lisp interpreter on a Raku grammar.

GRAMMAR · EVAL

Forth

Stack machine, also grammar-parsed.

GRAMMAR

Markdown → HTML

Converter — also runs in the browser.

REGEX · WASM

JSON

Parser + formatter, browser-ready too.

GRAMMAR · WASM

Pastebin

HTTP service on raw sockets.

SOCKETS

Chat server

Concurrent, many-client.

THREADS

Key-value store

Its own wire protocol.

SOCKETS

rakus

A static-file HTTP server.

SOCKETS

Plus three interpreters for *other* languages, each written in Raku: **JavaScript/TypeScript**, **Perl 5** (Raku parsing its own ancestor), and **Python 3** — off-side rule included, via a CPython-style INDENT/DEDENT tokenizer — with examples byte-identical to `node`, `perl` and `python3`. Every one of the 24 bundled `examples/` also compiles natively with `--exe`, byte-for-byte identical.

Modules from the ecosystem, **as installed.**

Raku++ installs modules with **its own installer** — `rakupp install JSON::Fast`, a checkout, or a URL — into the **same store zef uses**. A distribution installed by either tool is found by `use` under either engine.

THE BAR

48 / 59 distributions

in the per-release battery pass *their own* test suites — the files zef runs at install time, not a one-line API probe. It is the per-release QA instrument, not the ecosystem picture — the sweep is that.

THE SET

Top 50 by reverse-deps

JSON::Fast, XML, YAMLish, URI, File::Temp, File::Find, Terminal::ANSIColor, Digest, Encode — ranked over the ecosystem archive, version-pinned.

THE METHOD

Every fix is general

No module accommodations: `is raw` write-back, `CALL-ME` by name, `method FALLBACK`, `END` at process exit — all landed as interpreter fixes.

Far end of the same road: a live `HTTP/1.1 200 OK` over TLS — Cro's client on `I0::Socket::Async::SSL`, talking to the system OpenSSL through Raku++'s own NativeCall. Getting there was a chain of ~13 bugs, and nearly every one was a general correctness bug.

— BEYOND WHAT ROAST'S UNIT GRANULARITY EXERCISES

Real applications run **unmodified**.

RAKU-COURSE

~1,500 pages, byte-identical

The full course static-site generator regenerates the entire site byte-for-byte identically to Rakudo — including the real zef-installed

`YAMLish` module.

COVID.OBSERVER

End-to-end build

The site builder runs start to finish on Raku++ — a genuine data-processing application, not a curated snippet.

```
# exact arithmetic, lazy infinite sequence, grapheme strings –
# the things that make real programs match, not just tests
say 0.1 + 0.2 - 0.3;           # 0 (exact Rat)
say (1, 2, *+* ... *)[^10];    # (1 2 3 5 8 13 21 34 55 89)
say "café".chars;              # 4 (graphemes, not bytes)
```

More than a way to **run a file**.

V3.28.0

RakuAST, end to end

`.AST`, `.DEPARSE`, `.EVAL`, `visit-children`, `.rakudoc` — built as a **view** over the existing parse, not a second front end, so the ordinary path pays nothing for it: worst +2.7%, mean **-1.3%** over sixteen kernels.

V3.28.0

`--fmt`

A formatter with gofmt's promise — zero configuration, run it on every save. Whitespace only, and **three gates before a byte is written**: the input must parse, the result must be the same program, and formatting it again must change nothing.

V3.28.0

`use L10N::DE;`

...and the rest of the file is German. **Eleven languages**, off the same table RakuAST already reads. Upstream this is a slang; here it is a rewrite of the token stream, which is why it needs no flag when Rakudo wants `RAKUDO_RAKUAST=1`.

TARGETS

`--target=js`

Transpile a program to JavaScript for Node, Bun, Deno or a browser — with `--verify` to check the result against the interpreter that produced it.

PROTOCOLS

MCP · Jupyter · LSP

The interpreter over the **Model Context Protocol**, so an AI agent gets a Raku session with exact arithmetic and grammars as structured extraction. A **Jupyter kernel** that speaks ZMQ and signs its own messages: no ZeroMQ, no Python module. And `--lsp`.

EMBEDDING

Six host languages

`librakupp` behind a small C ABI, wrapped for **Python, JavaScript, Go, Rust, C++ and Wolfram**. Each gets the same two things: run Raku, and parse with Raku grammars where the grammar stays a `.raku` file.

Honest note: the **bindings and the MCP server are committed and gated on every push, but not announced** — they are in the repository so they cannot be lost, not because they have settled. Everything else on this slide shipped in a tagged release.

Not there yet.

- ▶ **Slangs** — pluggable syntax. There is no grammar object to mix a role into, and nine seams would have to grow one; the plan is written, the emulation route was rejected. **RakuAST shipped in v3.28.0** and was not the blocker.
- ▶ **C structs passed by value**, and callbacks fired from a thread the C library owns. The rest of NativeCall landed: `libffi` found at run time with nothing to install, plus structs, unions, pointers, callbacks and variadics.
- ▶ **Regex code blocks under backtracking** — `/(\d) { say $0 }/` re-fires its side effects; the fix has to defer them to the accepted path.
- ▶ **1,000 ecosystem distributions**, from 824. The lever is the 274 that never ran their own tests at all because a dependency failed first, and the 168-dist network/async cluster inside them.
- ▶ **The objects kernel** — Rakudo leads it by 1.6× compiled and 2.3× interpreted, and it measures the shape most real Raku is written in. Method calls and attribute reads stopped allocating in v3.24.0; nothing is profiled beyond that yet.
- ▶ A **bytecode VM**. Walking the AST costs ~115 ns a node against ~5 ns for JIT-compiled bytecode, and that ~23× is most of what separates Raku++ from Rakudo on tight per-character loops. Closing it is a different implementation, not an optimisation.

The cheap Roast wins are spent: moving the count now takes whole features — parse, runtime and dispatch together. The v3.26.0 source review reached **three files of eighty-three**, so that is open too.

— MACOS · LINUX · WINDOWS · THE BROWSER

Try **Raku++** today.

```
$ curl -fsSL https://raku.online/install.sh | sh
```

```
$ rakupp upgrade # later, in place
```

```
➤ raku.online # or run it in the browser, no install at all
```

github.com/ash/rakuppraku.onlineraku.online/specraku.online/tour

Neither installer needs root. On Windows, `irm ../install-windows.ps1 | iex` or a setup wizard with an Add/Remove Programs entry; on macOS, a Homebrew tap as well. Prebuilt archives on every GitHub Release — macOS universal, Linux x86_64 and ARM64, Windows x64, OpenBSD — plus the WebAssembly bundle. No runtime dependencies, anywhere.